DATA 260 Homework 6 Report

Sreeram Achutuni

SJSU ID: 019151808

GitHub repository: https://github.com/sriram2930/Data260-1808

Section 0: Config values

SID4	1808
PORT_BASE	8008
PREFIX	s1808
SEED	1808
VERIFY_SEED	261808
DOMAIN_ID	0, Campus course catalogue and enrolment
Hardware	Intel Core i7-11390H, 16GB RAM, integrated Iris Xe graphics (no dedicated GPU)
Local model	qwen2.5:1.5b-instruct via Ollama (generation), sentence-transformers/all-MiniLM-L6-v2 (episodic + duplicate-detection embeddings)
Repo	https://github.com/sriram2930/Data260-1808
Collaborators	Sbnikitha and supriyaselvanganesan have repo access
Commit tagged as hw6	29ff8ba437e101bb30c5e976f00b760ac5c99528

Note: evidence in this report is real captured terminal/API output (curl, mongosh, python test scripts run against the live backend, database, and local model), not GUI screenshots (Postman, MongoDB Compass) -- this homework was built and verified end to end without a human driving those particular GUIs, the same substitution used for MySQL evidence in HW4/HW5 when a Compass-equivalent wasn't in the loop.

Part 1: MongoDB Document API

Builds on HW5 with a MongoDB-backed document view of the campus course catalogue domain: an announcements collection (course update/enrollment/event notices) in a new database, s1808_doc, alongside the existing MySQL s1808_rel tables. Fields: title (required, max 100 chars), description (optional), status (enum draft/published/archived, default draft), category (enum course_update/enrollment/event/general), event_date (required), created_at/updated_at (automatic). Validation is enforced twice, independently: Pydantic at the API boundary and a real MongoDB $jsonSchema validator on the collection itself.

Code: Pydantic schema and MongoDB validator

```
class AnnouncementIn(BaseModel):
    title: str = Field(min_length=1, max_length=100)
    description: Optional[str] = None
```

```
    status: Status = "draft"
    category: Category
    event_date: datetime.datetime

ANNOUNCEMENT_SCHEMA = {
    "$jsonSchema": {
        "bsonType": "object",
        "required": ["title", "status", "category", "event_date", "created_at",
"updated_at"],
        "properties": {
            "title": {"bsonType": "string", "maxLength": 100},
            "status": {"bsonType": "string", "enum": ["draft", "published",
"archived"]},
            "category": {"bsonType": "string", "enum": ["course_update", "enrollment",
"event", "general"]},
            "event_date": {"bsonType": "date"},
        },
    }
}
```

CRUD verified live (output below)

```
--- POST (create, valid) ---
{"id":"6ad5e3d413482f9b4c0c8948","title":"Midterm date moved",...}
201

--- POST (missing required title) ---
{"detail":[{"type":"missing","loc":["body","title"],"msg":"Field required",...}]}
422

--- POST (invalid enum) ---
{"detail":[{"type":"literal_error","loc":["body","status"],
  "msg":"Input should be 'draft', 'published' or 'archived'",...}]}
422

--- GET (by id, after creating a 2nd real announcement) ---
{"id":"...","title":"Fall enrollment opens Nov 1","status":"published",
 "category":"enrollment","event_date":"2026-11-01T08:00:00",...}
200

--- PUT (update) ---
{"id":"...","title":"Midterm date moved again","status":"archived",...}
200  (updated_at changed, created_at unchanged)

--- GET by bad id ---
{"detail":"Announcement not found"}
404

--- DELETE ---
{"ok":true,"deleted_id":"..."}
200
```

MongoDB-level validator rejects bad data independently of the API

```
> db.announcements.insertOne({title:"ok title", status:"not_valid_enum", ...})
MongoServerError: Document failed validation
```

Collection contents (mongosh)

```
[
  {
    _id: ObjectId('6ad5e3ee13482f9b4c0c8949'),
    title: 'Fall enrollment opens Nov 1',
    description: 'Priority enrollment window for continuing students.',
    status: 'published',
    category: 'enrollment',
    event_date: ISODate('2026-11-01T08:00:00.000Z'),
    created_at: ISODate('2026-10-19T09:33:34.672Z'),
    updated_at: ISODate('2026-10-19T09:33:34.672Z')
  }
]
```

Part 2: Short-, Long-, and Episodic Memory

New collections in the same s1808_doc database: messages, summaries, episodes. POST /api/chat saves the user message, pulls the last SHORT_TERM_N messages, the latest session/lifetime summaries, and the top-k episodic facts (cosine similarity over all-MiniLM-L6-v2 embeddings), composes a prompt, and calls the local Ollama model through the HW1 ModelClient adapter. A session summary is generated every SUMMARIZE_EVERY_USER_MSGS=5 user messages and condensed into a lifetime summary once a user has at least 2 session summaries. A 12-message real conversation (run_chat_demo.py) exercised all three memory types in one run.

Code: the three endpoints

```
@router.post("/chat", response_model=ChatOut)
def chat(payload: ChatIn):
    result = chat_turn(payload.user_id, payload.session_id, payload.message,
SHORT_TERM_N)
    return {"reply": result["reply"], "session_id": result["session_id"], "used":
result["used"]}

@router.get("/memory/{user_id}")
def get_memory(user_id: str):
    db = get_db()
    messages = list(db.messages.find({"user_id": user_id}, {"_id": 0})
                    .sort("created_at", -1).limit(16))
    messages.reverse()
    session_summary = db.summaries.find_one(
        {"user_id": user_id, "scope": "session"}, {"_id": 0}, sort=[("created_at", -
1)])
    lifetime_summary = db.summaries.find_one(
        {"user_id": user_id, "scope": "user"}, {"_id": 0}, sort=[("created_at", -1)])
    episodes = list(db.episodes.find({"user_id": user_id}, {"_id": 0, "embedding": 0})
                    .sort("created_at", -1).limit(20))
```

```
    return {"messages": messages, "latest_session_summary": session_summary,
            "latest_lifetime_summary": lifetime_summary, "episodes": episodes}

@router.get("/aggregate/{user_id}")
def get_aggregate(user_id: str):
    pipeline = [{"$match": {"user_id": user_id}},
                {"$group": {"_id": {"$dateToString": {"format": "%Y-%m-%d", "date":
"$created_at"}},
                            "count": {"$sum": 1}}}, {"$sort": {"_id": 1}}]
    daily_counts = [{"date": r["_id"], "count": r["count"]} for r in
db.messages.aggregate(pipeline)]
    recent_summaries = list(db.summaries.find({"user_id": user_id}, {"_id": 0})
                            .sort("created_at", -1).limit(5))
    return {"daily_message_counts": daily_counts, "recent_summaries": recent_summaries}
```

Code: episodic extraction and retrieval

```
def extract_episodes(user_id, session_id, user_message):
    """Extracts up to 3 short durable facts via the local model, embeds each, stores
them."""
    prompt = (
        "Extract up to 3 short, durable facts about the user from this message, each
with "
        "an importance score from 0 to 1. Reply with ONLY a JSON array like "
        '[{"fact": "...", "importance": 0.8}]'. ' + f"Message: {user_message!r}"
    )
    result = get_model_client().complete([{"role": "user", "content": prompt}],
format="json")
    try:
        parsed = json.loads(result["content"])
        if isinstance(parsed, dict):
            # the small model sometimes returns one fact object directly
            parsed = [parsed] if "fact" in parsed else parsed.get("facts", [])
    except (json.JSONDecodeError, AttributeError):
        parsed = []
    # ... embed each fact.get("fact") and db.episodes.insert_one(...)

def retrieve_episodes(user_id, query_text, k=3):
    episodes = list(db.episodes.find({"user_id": user_id}))
    query_vec = np.array(embed_model.get_text_embedding(query_text))
    scored = [(cosine(query_vec, np.array(e["embedding"])), e) for e in episodes]
    scored.sort(key=lambda x: x[0], reverse=True)
    return [ep for _sim, ep in scored[:k]]
```

Conversation excerpt (full 12-turn run in RUN_LOG.txt)

```
--- turn 1 (49.2s) ---
user: Hi, my name is Sreeram and I am a grad student in Data Science.
assistant: Hello Sreeram! Welcome to the Data Science program...
used: short_term_count=1, has_session_summary=False, has_lifetime_summary=False,
episodic_facts=[]

--- turn 6 (24.4s) ---
user: What is the difference between a session summary and a lifetime summary?
```

```
assistant: A session summary is a brief summary of a specific event...
used: short_term_count=11, has_session_summary=True, has_lifetime_summary=False,
      episodic_facts=["My advisor's title is Course Advisor", "The user's name is
Sreeram...", ...]

--- turn 9 (15.5s) ---
user: What was the first thing I told you about myself?
assistant: I was informed that you are a grad student in Data Science.

--- turn 10 (57.5s) ---
user: Do you remember my preferred office hours format?
assistant: Yes, I remember that you prefer online office hours.

--- turn 11 (19.4s) ---
user: One more thing, I am also minoring in Business Analytics.
used: ..., has_lifetime_summary=True, ...
```

Note: by turn 7, the episodic store had already picked up one wrong fact ('Your major is Computer Science', when the user had said Data Science), and it kept resurfacing in later turns alongside correct facts. extract_episodes does not re-verify a fact against the conversation before storing it, so one hallucinated extraction becomes a persistent, retrievable memory from then on. This is a real property of the system as built, documented here rather than fixed, since catching it required reading the raw per-turn output, not just the final answer.

GET /api/memory/u1 (excerpt)


```
{
  "messages": [ ... last 16 messages ... ],
  "latest_session_summary": {"scope":"session","text":"- Sreeram is a grad
student...",...},
  "latest_lifetime_summary": {"scope":"user","session_id":null,
    "text":"- Sreeram is a grad student in Data Science.\n- Courses include...\n-
Sreeram prefers online office hours...",...},
  "episodes": [ ... last 20 episodic facts, no embedding vectors ... ]
}
```


GET /api/aggregate/u1


```
{
  "daily_message_counts": [{"date": "2026-10-19", "count": 24}],
  "recent_summaries": [
    {"scope":"user","session_id":null,"text":"- Sreeram is a grad student in Data
Science.\n..."},
    {"scope":"session","text":"- **User Information**: Sreeram, a grad
student...\n..."},
    {"scope":"session","text":"- Sreeram is a grad student in Data Science.\n..."}
  ]
}
```


MongoDB collections (mongosh)

```
> db.messages.countDocuments({})
24
```

```
> db.summaries.find({},{text:1,scope:1,session_id:1})
{ session_id: "962b885ce775", scope: "session", text: "- Sreeram is a grad student..."
}
{ session_id: "962b885ce775", scope: "session", text: "- **User Information**:
Sreeram..." }
{ session_id: null, scope: "user", text: "- Sreeram is a grad student in Data
Science...\n- Sreeram prefers online office hours for better communication...\n- Data
Science and AI departments offer courses..." }

> db.episodes.find({},{embedding:0}).limit(3)
{ fact: "The user's name is Sreeram and they are a grad student in Data Science.",
importance: 0.8 }
{ fact: "The Computer Science department offers courses in programming languages...",
importance: 0.8 }
{ fact: "The user is interested in cloud computing.", importance: 0.8 }
```

Part 3: What Memory Costs

The same fixed 30-message conversation (planted facts: name/student id/favorite subject in messages 1-3, birthplace in message 16) ran in a fresh session at SHORT_TERM_N = 4, 12, 30, then once more at 12 with an eviction/compression policy (short-term message truncation to 120 chars), asking memory_probes.yaml's five questions after each run.

Setting	Mean prompt tokens/turn	Mean latency (ms)	Probe quality (1-5)
SHORT_TERM_N=4	1790.3	25117.7	5.00
SHORT_TERM_N=12	1156.1	18149.8	4.60
SHORT_TERM_N=30	1433.3	18095.7	5.00
Eviction/compression (N=12)	728.7	6535.5	not measured (see note)

Note: the eviction/compression run was cut off at turn 26/30 (no probes) due to real-world time constraints finishing this homework on a CPU-only local model. The 26-turn average is still a fair token/latency comparison (an average isn't biased by sample length the way a total would be), but the probe-quality cell is genuinely missing data, not a zero.

SHORT_TERM_N barely affects probe quality on its own (4.6-5.0 across all three raw window sizes), because episodic memory and the session/lifetime summaries carry the planted facts forward independently of the raw window. Even SHORT_TERM_N=4, which drops the planted facts from the window after message 5, scored a perfect 5/5. The eviction/compression policy cut mean prompt tokens from 1156 to 729 (-37%) and mean latency from 18.1s to 6.5s (-64%) over the portion measured, a larger latency drop than the token drop alone predicts, consistent with this model slowing disproportionately as prompts approach its 4096-token window.

Memory-cost estimate (10,000 conversations x 30 turns)

Using $1/M input tokens, $5/M output tokens, each setting's mean prompt tokens/turn, and ~100 output tokens/turn observed across settings:

Setting	Input cost	Output cost	Total
SHORT_TERM_N=4 (worst)	$537.09	$150.00	$687.09
Eviction/compression (best)	$218.61	$150.00	$368.61

Worst is the highest mean-tokens setting with no mitigation (SHORT_TERM_N=4, counterintuitively the smallest raw window, since summaries and episodic facts end up dominating the prompt more than the window itself once several summarization cycles have run). Best is the only setting that actively reduces token cost rather than picking a different window size. The gap ($687 vs $369 for the same 300,000 turns) is driven entirely by input tokens; a policy that compresses the prompt is worth roughly 1.9x in cost at this volume, independent of answer quality.

Part 4: File-Based Memory and Consolidation

AGENT.md (reused and expanded from HW1) now covers five categories: project description, technology stack, coding conventions, testing approach, prohibited actions. MEMORY.md and file_memory.py implement loading, the consolidation trigger check (>40 non-comment lines, or >24h since last consolidation), and the /memory, /compact, /dream, /help commands (memory_cli.py).

Consolidation design: three attempts

```
1. One-shot full rewrite: asked the model to merge duplicates and resolve
contradictions
   in the whole 56-line test file at once. Barely touched anything: 56 -> 54 lines,
none
   of the 3 planted contradictions were resolved.

2. One-shot JSON line-number classification: asked the model to find duplicate groups
and
   contradiction pairs by line number instead of rewriting prose. Found zero real
   duplicates and hallucinated one false contradiction between unrelated facts.

3. Final design: embeddings find candidate pairs "about the same thing"; a
deterministic
   check (differing numbers, or an antonym like allowed/blocked) decides contradiction
vs.
   duplicate; a narrow yes/no model call confirms each duplicate candidate before
dropping
   anything; each contradiction gets its own small, focused merge call.
```

Stress test: 56-line file, 3 planted duplicates, 3 planted contradictions

	Result
Lines before	56
Lines after (pipeline's raw output)	51
Contradictions found	3 of 3 planted
Contradictions resolved correctly	2 of 3 (port 8008; Department deletion blocked)

Contradiction resolved INCORRECTLY	SHORT_TERM_N: pipeline said "8 is current, was 12" - backwards. Real default in memory.py is 12.
Duplicate merges that lost information	1-2 (MCP tool names, retry-cap detail dropped in merges)

Note: the pipeline's raw 51-line output still didn't meet the assignment's <40 target. The real reason: only 6 of 56 lines were truly redundant (3 dup pairs + 3 contradiction pairs); even perfect dedup caps out around 50 lines, since the other 44 are genuinely distinct facts. Getting under 40 required a human pass grouping related-but-distinct facts into combined bullets, a step the automated pipeline doesn't attempt. The human-corrected final version (test_memory_after.md) is 26 lines: fixes the one incorrect resolution (states the true SHORT_TERM_N=12, with the pipeline's wrong answer left visible in a parenthetical so the correction is auditable, not a silent overwrite), restores the two lost details, and groups facts into combined bullets. No fact from the original 56 lines is missing from the final 26.

AGENT.md compliance check

```
[PASS] has a docstring (AGENT.md: short module-level docstring)
[PASS] does not import ollama directly (AGENT.md: always via ModelClient)
[PASS] does not open a new MongoClient (AGENT.md: reuse existing adapters)

overall: PASS

Generated function (for reference):
def get_published_announcements(db):
    """Returns the titles of the 3 most recently created and published
announcements."""
    announcements = db.announcements.find_and_modify(
        sort=[(-1, "created_date")], limit=3, new=True).pretty()
    return [a["title"] for a in announcements]
```

Comparing MongoDB memory and file-based memory

	MongoDB-backed (Part 2)	File-based (Part 4)
Inspectability	Needs a Mongo client; structured/queryable, not human-readable at a glance	Plain Markdown, readable/editable anywhere, no tooling
Persistence after restart	Durable as long as the container/volume persists	Durable as long as the files aren't deleted; no schema protects a bad manual edit
Scalability across sessions	Scales well: indexed by user_id, cheap to query out of millions of docs	Does not scale: one flat file per project, not per user/session
Facts that must not be forgotten	Good, but only as safe as the query that retrieves it	Better for a small curated set (whole file always in the prompt); doesn't scale to large sets

MongoDB memory is the right fit for high-volume, per-user, queryable history (Part 2's actual job); file-based memory is the right fit for a small number of durable, project-wide

facts that should always be present (exactly what AGENT.md/MEMORY.md are used for here).

AI Use Statement

1. What I used an AI assistant for, and what I did myself

I used Claude Code to design and write the MongoDB schema and CRUD API, the short/long/episodic memory system and chat endpoint, the token-cost experiment harness, and the file-based memory system with consolidation. It ran all of this against real infrastructure: a real MongoDB container, the real local Ollama model, and real embeddings.

What I did myself: designed the announcements entity, decided how the one-turn lag in summaries/episodic retrieval should work (confirmed intentional, not a bug, by reading the actual turn-by-turn output), picked the 30-message test conversation and where to plant probe facts, iterated through three consolidation designs myself before the one that works, and manually reviewed and corrected the consolidation pipeline's one factually wrong resolution rather than accepting it at face value.

2. One AI-produced output that was wrong/unsuitable

Two. Episode extraction silently stored nothing on the first real conversation turn: the model replied with a single JSON object (the fact itself) instead of a list, and my parsing code treated that as "no facts." Separately, Part 4.5's stress test expects consolidation to merge 3+ duplicates and resolve 3+ contradictions in a 56-line file and land under 40 lines; the first design (one-shot rewrite) went from 56 to 54 lines and resolved none of the three planted contradictions.

3. How I detected the problem

For the episode bug: queried MongoDB directly after the first turn and got zero results despite an obviously memorable first message, then ran the extraction prompt by hand to see the model's raw (unwrapped) reply. For consolidation: the before/after line counts (56 to 54) immediately signaled something had gone wrong, and reading the actual file content confirmed all three planted contradictions were still present, untouched.

4. What I changed and why it works now

For episodes: the parser now checks whether a returned dict looks like a single fact directly (has a "fact" key) and wraps it in a list, in addition to the {"facts": [...]} case. For consolidation: after two more failed one-shot designs, duplicates are now found via embedding similarity and confirmed with a narrow yes/no call before anything is dropped, contradictions are found by checking which similar pair differs in a number or a polarity word, and only the final one-line merge for a confirmed contradiction goes through the model. This caught all three planted contradictions reliably, though it still got one resolution factually wrong (SHORT_TERM_N) and missed two of three duplicate pairs due to the

similarity threshold - documented and corrected by hand in METRICS.md and test_memory_after.md rather than quietly accepted.